

Contents

1	Introduzione e Definizione del Problema	3
2	Modellazione dei Dati e Integrazione Cloud	3
2.1	Le Entità Software	3
2.2	Perché Supabase?	3
3	Logica di Scoring e Prioritarizzazione	4
3.1	Priorità Individuale (Person)	4
3.2	Priorità Globale dell'Ente (Beneficiary)	4
4	L'Algoritmo Genetico (GA) Cooperativo	4
4.1	Rappresentazione del Cromosoma	4
4.2	Operatori di Selezione, Crossover e Mutazione	5
4.3	Meccanismo di Stochastic Repair	5
4.4	Funzione di Fitness	5
5	Fase di Post-Processing: L'Approccio Memetico	6
6	Analisi delle Scelte Progettuali: "Perché Sì" e "Perché No"	6
7	Estensioni Future e Miglioramenti	7
7.1	Ottimizzazione Multi-Obiettivo Reale (NSGA-II)	7
7.2	Proposta di Nuove Visualizzazioni e Grafici	8
7.3	Proposta di Nuove Tabelle Comparative	8

1 Introduzione e Definizione del Problema

Il progetto risponde all'esigenza di ripartire in modo efficiente, equo e mirato un insieme eterogeneo di prodotti alimentari, stoccati in un magazzino centrale, a una serie di enti beneficiari (quali Ospedali, Mense, Case Famiglia).

Ogni ente esprime un **fabbisogno calorico totale** calcolato sulla base delle specifiche persone fisiche ad esso associate. Gli obiettivi fondamentali dell'ottimizzazione sono:

1. **Massimizzazione dell'equità:** Minimizzare il divario tra le calorie richieste da un ente e quelle effettivamente allocate, pesando l'errore in base a un indice di urgenza dinamico.
2. **Soddisfacimento dei vincoli fisici (Hard Constraints):** Garantire che la quantità totale di prodotti allocati non superi mai le scorte fisicamente disponibili in magazzino.
3. **Prevenzione dello spreco:** Evitare contestualmente il sotto-provvigionamento delle strutture critiche e il sovra-provvigionamento inutile (surplus calorico).

2 Modellazione dei Dati e Integrazione Cloud

Il sistema adotta un'architettura disaccoppiata in cui lo stato iniziale del mondo è memorizzato su un database relazionale gestito tramite **Supabase**. L'interfaccia con l'algoritmo avviene mediante un modulo di mapping Object-Relational implicito scritto in Python.

2.1 Le Entità Software

Il dominio è modellato attraverso tre classi fondamentali (**dataclasses**):

- **Beneficiary:** Rappresenta l'ente destinatario. È caratterizzato da un `id`, un `name` e un `type` (Mensa, Ospedale, Famiglia, Casa Famiglia).
- **Person:** Rappresenta il singolo individuo afferente a un ente. Contiene attributi anagrafici e nutrizionali fondamentali (`age`, `gender`, `caloric_requirement`, `classification`). Calcola a runtime la propria priorità individuale.
- **Product:** Modella il bene alimentare con il rispettivo inventario (`quantity`) e il profilo macro-nutrizionale (`calories`, `fats`, `salts`, `sugars`, `proteins`).

2.2 Perché Supabase?

La scelta di Supabase come backend BaaS (Backend-as-a-Service) rispetto a un database locale (es. SQLite) o a file CSV statici è motivata da requisiti di scalabilità e persistenza cooperativa:

- **Vantaggi:** Consente l'aggiornamento in tempo reale delle scorte o dei beneficiari da interfacce esterne (web o mobile), supporta nativamente vincoli di integrità referenziale relazionale tramite PostgreSQL ed evita il disallineamento dei dati tra diverse istanze di esecuzione dell'algoritmo.

3 Logica di Scoring e Prioritarizzazione

Per garantire che le risorse vengano distribuite secondo criteri di giustizia sociale ed emergenziale, è stato implementato un sistema di prioritarizzazione a due livelli nel modulo `priorities.py`.

3.1 Priorità Individuale (Person)

Ogni individuo riceve un punteggio di priorità formulato come combinazione lineare pesata di quattro indicatori normalizzati:

$$P_{\text{persona}} = w_{\text{età}} \cdot S_{\text{età}} + w_{\text{ente}} \cdot S_{\text{ente}} + w_{\text{cal}} \cdot S_{\text{cal}} + w_{\text{genere}} \cdot S_{\text{genere}} \quad (1)$$

Con i pesi fissati empiricamente a: $w_{\text{età}} = 0.40$, $w_{\text{ente}} = 0.30$, $w_{\text{cal}} = 0.15$, $w_{\text{genere}} = 0.15$.

- Il punteggio premia le fasce d'età vulnerabili (neonati ≤ 3 anni e anziani ≥ 65 anni ricevono il massimo score).
- Il tipo di ente discrimina il contesto operativo, assegnando priorità massima ai degenti ospedalieri ($S_{\text{ente}} = 10$) e alle mense dei poveri ($S_{\text{ente}} = 8$).

3.2 Priorità Globale dell'Ente (Beneficiary)

La mappa delle priorità degli enti (`ben_priorities_map`) aggrega i punteggi delle persone associate:

$$P_{\text{ente}} = \frac{1}{N} \sum_{i=1}^N P_{\text{persona},i} + \text{Fattore Correttivo}(N) \quad (2)$$

Questo indicatore agisce direttamente come moltiplicatore di penalità all'interno della funzione di fitness del GA: più un ente è prioritario, maggiore sarà la penalizzazione algoritmica nel caso in cui il suo fabbisogno rimanga insoddisfatto.

4 L'Algoritmo Genetico (GA) Cooperativo

Il cuore computazionale della ricerca globale risiede in `genetic_algorithm.py`. Data la natura combinatoria del problema, l'algoritmo esplora in modo efficiente lo spazio delle possibili assegnazioni.

4.1 Rappresentazione del Cromosoma

Il cromosoma è stato modellato come una struttura dati gerarchica sparsa basata su dizionari annidati:

$$\text{Dict}[\text{int}, \text{Dict}[\text{int}, \text{int}]] \rightarrow \{\text{id_beneficiario}: \{\text{id_prodotto}: \text{quantità_allocata}\}\}$$

Perché questa struttura? (Perché Sì / Perché No):

- **Perché Sì:** Rispetto a una matrice bidimensionale fissa ($\text{Enti} \times \text{Prodotti}$), il dizionario sparso riduce drasticamente l'occupazione di memoria quando molti prodotti non sono assegnati a un determinato ente. Inoltre, l'accesso alle chiavi tramite ID preserva l'accoppiamento diretto con i vincoli del database relazionale, facilitando le operazioni di mutazione locale.
- **Perché No:** Una codifica a stringa binaria piatta avrebbe richiesto una decodifica computazionalmente onerosa a ogni ciclo di fitness ed espresso soluzioni per la maggior parte invalide o fuori dai limiti.

4.2 Operatori di Selezione, Crossover e Mutazione

- **Selezione per Torneo ($k = 3$):** Evita la convergenza prematura rispetto alla selezione a ruota di roulette (proporzionale alla fitness), mantenendo costante la pressione selettiva anche in presenza di variazioni di scala nei punteggi di fitness.
- **Crossover a Taglio Singolo:** Scambia interi blocchi di allocazioni di beneficiari tra due genitori. Ciò preserva le combinazioni di allocazione locali che si sono dimostrate efficaci per determinati enti.
- **Mutazione Adattativa:** Altera stocasticamente la quantità di un prodotto per un ente, consentendo al sistema di esplorare nuove zone dello spazio di ricerca ed evitare minimi locali.

4.3 Meccanismo di Stochastic Repair

Una delle componenti più innovative è la funzione `repair_chromosome`. Se una soluzione generata da crossover o mutazione viola il vincolo di scorta massima del magazzino, la funzione interviene attivamente ripristinando la validità:

```

1 for prod_id, total_qty in total_allocated.items():
2     available = product_map[prod_id].quantity
3     if total_qty > available:
4         excess = total_qty - available
5         while excess > 0:
6             eligible_bens = [b_id for b_id, allocs in chromosome.items()
7                             if allocs.get(prod_id, 0) > 0]
8             if not eligible_bens: break
9             target_ben = random.choice(eligible_bens)
10            chromosome[target_ben][prod_id] -= 1
11            excess -= 1

```

Listing 1: Logica del Repair stocastico del magazzino

Questo operatore converte un vincolo rigido (*hard constraint*) in una ricerca guidata, impedendo all'algoritmo di sprecare valutazioni di fitness su individui impossibili da attuare nella realtà.

4.4 Funzione di Fitness

La funzione di fitness (`fitness.py`) valuta la bontà di un cromosoma partendo da un valore base ideale (1.000.000) e applicando penalità cumulative decrescenti:

$$\text{Fitness} = 10^6 - \sum_{p \in P} (\text{Eccesso}_p \cdot 5000) - \sum_{b \in B} (|\text{CalorieAllocated}_b - \text{Target}_b| \cdot 0.5 \cdot P_b) \quad (3)$$

Il coefficiente 5000 applicato al superamento delle scorte funge da barriera invalicabile, costringendo la popolazione a convergere verso soluzioni fisicamente realizzabili.

5 Fase di Post-Processing: L'Approccio Memetico

Sebbene l'Algoritmo Genetico sia eccellente nel trovare la struttura globale ottima della distribuzione, a causa della sua natura stocastica può faticare a esaurire con precisione chirurgica le rimanenze unitarie del magazzino o a stabilizzare perfettamente le oscillazioni attorno al fabbisogno esatto.

Per risolvere questo problema, in `exports.py` è stata introdotta la funzione `apply_greedy_cleanup`. Questo modulo agisce come una fase di ricerca locale deterministica (trasformando il GA in un **Algoritmo Memetico**):

- **Logica:** Prende i prodotti residui in magazzino e tenta di allocarli in modo sequenziale (*greedy*) partendo dagli enti a priorità più alta.
- **Hard Constraint sul Surplus (Blocco Rigido):** L'ottimizzazione locale introduce una regola ferrea: l'assegnazione supplementare viene interrotta immediatamente se porta le calorie totali dell'ente a superare il 100% del suo fabbisogno. Questo impedisce di sprecare cibo accumulando surplus su un singolo ente a discapito di altri.

6 Analisi delle Scelte Progettuali: "Perché Sì" e "Perché No"

A beneficio dei revisori esterni, vengono qui esplicitate le motivazioni ingegneristiche che hanno guidato lo sviluppo, confrontandole con le alternative scartate.

Table 1: Analisi comparativa delle scelte architetturali e metodologiche.

Componente	Scelta Adottata (Perché Sì)	Alternativa (Perché No)	Scartata
Paradigma Algoritmico	Algoritmo Genetico Ibrido (Memetico): Gestisce relazioni non lineari, vincoli complessi e funzioni di priorità arbitrarie senza richiedere la differenziabilità.	Programmazione Intera (MILP): Sebbene garantisca l'ottimo globale, soffre di rigidità matematica nell'incorporare penalità nutrizionali non lineari e logiche di scoring dinamiche.	Lineare
Gestione Vincoli Magazzino	Repair Stocastico + Penalità: Corregge attivamente i cromosomi prima o durante la valutazione. Mantiene alta la densità di soluzioni valide nella popolazione.	Esclusione per Sola Penalità (<i>Death Penalty</i>): Scartare o penalizzare i cromosomi non validi senza ripararli rallenta la convergenza, poiché gran parte delle mutazioni casuali risulterebbe invalida.	
Risoluzione del Surplus	Post-Processing Greedy con Cap al 100%: Isola la complessità combinatoria dal GA, rifinando la soluzione in tempo polinomiale alla fine del processo.	Inserimento del Cap nel GA: Avrebbe ristretto eccessivamente lo spazio di ricerca del GA, impedendo l'esplorazione di soluzioni temporaneamente sovraccariche ma geneticamente promettenti.	

7 Estensioni Future e Miglioramenti

Il sistema attuale è robusto e funzionante, ma per elevare il livello di accuratezza accademica e completezza operativa si propongono i seguenti sviluppi.

7.1 Ottimizzazione Multi-Obiettivo Reale (NSGA-II)

Attualmente, i vincoli relativi a macro e micro-nutrienti (grassi, zuccheri, sali, proteine) sono estratti nei report ma non ottimizzati attivamente nella fitness, che si concentra prevalentemente sul bilancio calorico.

- **Miglioria:** Sostituire il GA mono-obiettivo con l'algoritmo **NSGA-II** (Non-dominated Sorting Genetic Algorithm II). Ciò consentirebbe di calcolare una **Frontiera di Pareto** in grado di bilanciare contemporaneamente:
 1. Minimizzazione del deficit calorico.
 2. Minimizzazione del surplus di zuccheri e sodio (particolarmente critico per il target "Ospedale").
 3. Massimizzazione dell'apporto proteico per i beneficiari in crescita (Case Famiglia).

7.2 Proposta di Nuove Visualizzazioni e Grafici

Per rendere la comparazione analitica impeccabile per stakeholder esterni, si suggerisce l'implementazione dei seguenti strumenti grafici nel modulo `exports.py`:

1. **Radar Chart Bilancio Nutrizionale (Grafico a Ragno):** Un grafico radar sovrapposto per ogni ente che metta a confronto il profilo nutrizionale ideale (Target Proteine, Grassi, Carboidrati) con quello effettivamente fornito dal piano di distribuzione.
2. **Mappa di Calore della Soddisfazione Individuale:** Una *Heatmap* organizzata per classi d'età e genere per identificare visivamente se l'algoritmo stia penalizzando involontariamente una specifica demografia a causa della disponibilità dei prodotti in magazzino.
3. **Grafico di Sensibilità dei Parametri del GA:** Una visualizzazione dell'andamento della convergenza al variare della dimensione della popolazione ($N \in \{50, 150, 300\}$) e del tasso di mutazione ($pm \in \{0.01, 0.05, 0.1\}$), per giustificare scientificamente l'iper-parametrizzazione scelta.

7.3 Proposta di Nuove Tabelle Comparative

Si raccomanda l'introduzione di una tabella di sintesi delle metriche aggregate per valutare l'efficienza dell'algoritmo rispetto a una baseline intuitiva (es. distribuzione puramente equiripartita):

Table 2: Struttura proposta per la tabella di validazione scientifica (Baseline vs GA Ibrido).

Metrica di Performance	Distribuzione Casuale	GA Puro	GA Ibrido (M)
Indice di Gini della Soddisfazione (Equità)	0.54	0.12	0.04
Percentuale Rimanenze Magazzino (%)	18.3%	4.1%	0.0%
Violazione dei Vincoli di Cap (# eventi)	12	0	0
Tempo di Calcolo (s)	0.01s	14.5s	14.9s